

## 1 MongoDB (50%)

1. Load the file restaurants.json into a MongoDB database. The file contains data on 3,772 restaurants in New York City. Write the following queries in MongoDB (show the output of your queries):

(a) Display all the restaurants located in the boroughs Bronx or Brooklyn.

### Solution:

```
db.restaurants.find(
  { borough: { $in: ["Bronx", "Brooklyn"]} }
)
```

(b) Find the restaurant id, name, borough and cuisine for those restaurants whose name starts with the letters 'Mad'.

### Solution:

```
db.restaurants.find(
  { name: { $regex: /^Mad/i } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
```

```
test> use restaurantsdb
switched to db restaurantsdb
restaurantsdb> db.restaurants.find(
...   { name: { $regex: /^Mad/i } },
...   { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
... )
{
  borough: 'Manhattan',
  cuisine: 'American',
  name: 'Madison Square',
  restaurant_id: '00000007'
},
{
  borough: 'Manhattan',
  cuisine: 'Indian',
  name: 'Madras Mahal',
  restaurant_id: '00019978'
},
{
  borough: 'Manhattan',
  cuisine: 'American',
  name: 'Madame X',
  restaurant_id: '00011024'
},
{
  borough: 'Manhattan',
  cuisine: 'French',
  name: 'Madison Bistro',
  restaurant_id: '00057000'
},
{
  borough: 'Brooklyn',
  cuisine: 'African',
  name: 'Madiba',
  restaurant_id: '00000101'
},
{
  borough: 'Bronx',
  cuisine: 'Italian',
  name: 'Madison 1',
  restaurant_id: '00021000'
},
{
  borough: 'Manhattan',
  cuisine: 'Hotdogs',
  name: 'Madame Tadamud's',
  restaurant_id: '00050007'
},
{
  borough: 'Manhattan',
  cuisine: 'American',
  name: 'Mad River Bar & Grille',
  restaurant_id: '00050224'
}
```

(c) Find the restaurants that have received a score between 80 and 90 (inclusive).

### Solution:

```
db.restaurants.find(
  { "grades.score": { $gte: 80, $lte: 90 } }
).pretty()
```

(d) Display the restaurant id and name of restaurants which have received a 'C' grade in year 2014.

### Solution:

```
db.restaurants.find(
  {
    "grades.grade": "C",
    "grades.date": { $gte: new ISODate("2014-01-01"), $lt: new ISODate("2015-01-01") }
  },
  { restaurant_id: 1, name: 1, _id: 0 }
).pretty()
```

```
restaurantsdb> db.restaurants.find(
...   {
...     "grades.grade": "C",
...     "grades.date": { $gte: new ISODate("2014-01-01"), $lt: new ISODate("2015-01-01") }
...   },
...   { restaurant_id: 1, name: 1, _id: 0 }
... ).pretty()
...
[
  { name: 'May May Kitchen', restaurant_id: '40358429' },
  { name: 'Terminal Cafe/Yankee Clipper', restaurant_id: '40364262' },
  { name: 'Texas Rotisserie', restaurant_id: '40364304' },
  { name: 'B & M Hot Bagel & Grocery', restaurant_id: '40364299' },
  { name: 'Old Town Bar & Restaurant', restaurant_id: '40364389' },
  { name: 'Polish National Home', restaurant_id: '40364404' },
  { name: 'Wyac Main Dining Room', restaurant_id: '40364467' },
  { name: 'Tout Va Bien', restaurant_id: '40364576' },
  { name: 'Marchis Restaurant', restaurant_id: '40364668' },
  { name: 'Shell Lanes', restaurant_id: '40365043' },
  { name: 'Como Pizza', restaurant_id: '40365288' },
  { name: 'The Princeton Club', restaurant_id: '40365361' },
  { name: 'Reynold'S Bar', restaurant_id: '40365423' },
  { name: 'King Yum Restaurant', restaurant_id: '40365592' },
  { name: 'Yankee Tavern', restaurant_id: '40365499' },
  { name: 'Nanni Restaurant', restaurant_id: '40366157' },
  { name: 'Emilio Iii Bar', restaurant_id: '40366214' },
  { name: 'Don Peppe', restaurant_id: '40366230' },
  { name: 'Dangerfield'S Night Club', restaurant_id: '40366381' },
  { name: 'The Punch Bowl', restaurant_id: '40366497' }
]
Type "it" for more
```

(e) Find the cuisine that has the highest number of restaurants.

### Solution:

```
db.restaurants.aggregate([
  { $group: { _id: "$cuisine", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
```

{ \$limit: 1 }

)

```
restaurantsdb> db.restaurants.aggregate([
...   { $group: { _id: "$cuisine", count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 1 }
... ])
[ { _id: 'American ', count: 2510 } ]
```

(f) Find the restaurants that do not prepare an 'American' cuisine and their average grade score is higher than 30. Display the restaurant ids and their average score.

**Solution:**

```
db.restaurants.aggregate([
  { $match: { cuisine: { $ne: "American" } } },
  { $unwind: "$grades" },
  { $group: {
    _id: "$restaurant_id",
    avgScore: { $avg: "$grades.score" }
  }},
  { $match: { avgScore: { $gt: 30 } } },
  { $project: { restaurant_id: "$_id", avgScore: 1, _id: 0 } }
```

)

```
restaurantsdb> db.restaurants.aggregate([
...   { $match: { cuisine: { $ne: "American" } } },
...   { $unwind: "$grades" },
...   { $group: {
...     _id: "$restaurant_id",
...     avgScore: { $avg: "$grades.score" }
...   }},
...   { $match: { avgScore: { $gt: 30 } } },
...   { $project: { restaurant_id: "$_id", avgScore: 1, _id: 0 } }
... ])
[
  { avgScore: 32.142857142857146, restaurant_id: '40366157' },
  { avgScore: 30.8, restaurant_id: '40825993' },
  { avgScore: 30.6, restaurant_id: '40624470' },
  { avgScore: 33.666666666666664, restaurant_id: '40372466' },
  { avgScore: 30.8, restaurant_id: '40374268' },
  { avgScore: 36, restaurant_id: '40756344' },
  { avgScore: 32.6, restaurant_id: '40387237' },
  { avgScore: 38.6, restaurant_id: '40393488' }
]
```

(g) For each restaurant display only the grades that were recorded from the year 2014 onwards.

**Solution:**

```
db.restaurants.find(
  {},
```

```

{
  "name": 1,
  "restaurant_id": 1,
  "grades": {
    "$filter": {
      "input": "$grades",
      "as": "grade",
      "cond": { "$gte": ["$$grade.date", new Date("2014-01-01")] }
    }
  },
  "_id": 0
}
)

```

(h) Calculate the average score across all the restaurants in the collection.

#### **Solution:**

```

db.restaurants.aggregate([
  { $unwind: "$grades" },
  { $group: { _id: null, avgScore: { $avg: "$grades.score" } } },
  { $project: { _id: 0, avgScore: 1 } }
])

```

```

restaurantsdb> db.restaurants.aggregate([
...   { $unwind: "$grades" },
...   { $group: { _id: null, avgScore: { $avg: "$grades.score" } } },
...   { $project: { _id: 0, avgScore: 1 } }
... ])
[ { avgScore: 11.427736743468195 } ]

```

2. Create a collection named sales with the following documents:

```

{ "_id": 1, "city": "Berkeley", "state": "CA", "qty": 648 },
{ "_id": 2, "city": "Bend", "state": "OR", "qty": 491 },
{ "_id": 3, "city": "Kensington", "state": "CA", "qty": 233 },
{ "_id": 4, "city": "Eugene", "state": "OR", "qty": 842 },

```

```
{ "_id": 5, "city": "Reno", "state": "NV", "qty": 655 },
{ "_id": 6, "city": "Portland", "state": "OR", "qty": 408 },
{ "_id": 7, "city": "Sacramento", "state": "CA", "qty": 574 }
```

### Solution:

```
db.sales.insertMany([
  { "_id": 1, "city": "Berkeley", "state": "CA", "qty": 648 },
  { "_id": 2, "city": "Bend", "state": "OR", "qty": 491 },
  { "_id": 3, "city": "Kensington", "state": "CA", "qty": 233 },
  { "_id": 4, "city": "Eugene", "state": "OR", "qty": 842 },
  { "_id": 5, "city": "Reno", "state": "NV", "qty": 655 },
  { "_id": 6, "city": "Portland", "state": "OR", "qty": 408 },
  { "_id": 7, "city": "Sacramento", "state": "CA", "qty": 574 }
])
```

```
restaurantsdb> db.sales.insertMany([
...   { "_id": 1, "city": "Berkeley", "state": "CA", "qty": 648 },
...   { "_id": 2, "city": "Bend", "state": "OR", "qty": 491 },
...   { "_id": 3, "city": "Kensington", "state": "CA", "qty": 233 },
...   { "_id": 4, "city": "Eugene", "state": "OR", "qty": 842 },
...   { "_id": 5, "city": "Reno", "state": "NV", "qty": 655 },
...   { "_id": 6, "city": "Portland", "state": "OR", "qty": 408 },
...   { "_id": 7, "city": "Sacramento", "state": "CA", "qty": 574 }
... ])
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5, '5': 6, '6': 7 }
}
```

(a) Find the total quantity of sales for each state and sort the quantities in descending order. The result should look as follows:

```
[
  { state: 'OR', total_qty: 1741 },
  { state: 'CA', total_qty: 1455 },
  { state: 'NV', total_qty: 655 }
]
```

### Solution:

```
db.sales.aggregate([
  { $group: { _id: "$state", total_qty: { $sum: "$qty" } } },
  { $sort: { total_qty: -1 } }
```

D)

```
restaurantsdb> db.sales.aggregate([
...   { $group: { _id: "$state", total_qty: { $sum: "$qty" } } },
...   { $sort: { total_qty: -1 } }
... ])
[
  { _id: 'OR', total_qty: 1741 },
  { _id: 'CA', total_qty: 1455 },
  { _id: 'NV', total_qty: 655 }
]
```

(b) Perform the following update operations on the sales collection:

i. Change the quantity of the sales in Berkeley from 648 to 750.

**Solution:**

```
db.sales.updateOne(
  { city: "Berkeley" },
  { $set: { qty: 750 } }
)
```

```
restaurantsdb> db.sales.updateOne(
...   { city: "Berkeley" },
...   { $set: { qty: 750 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

ii. Increase the quantity of all the sales in Oregon by 50.

**Solution:**

```
db.sales.updateMany(
  { state: "OR" },
  { $inc: { qty: 50 } }
)
```

```
restaurantsdb> db.sales.updateMany(
...   { state: "OR" },
...   { $inc: { qty: 50 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

iii. Add to sales no. 5 the names of the salespeople: David and Martha.

**Solution:**

```
db.sales.updateOne(
  { _id: 5 },
```

```
{ $set: { salespeople: ["David", "Martha"] } }
```

```
)
```

```
restaurantsdb> db.sales.updateOne(
...   { _id: 5 },
...   { $set: { salespeople: ["David", "Martha"] } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

iv. Add James to the list of salespeople of sales no. 5.

**Solution:**

```
db.sales.updateOne(
```

```
  { _id: 5 },
```

```
  { $addToSet: { salespeople: "James" } }
)
```

```
restaurantsdb> db.sales.updateOne(
...   { _id: 5 },
...   { $addToSet: { salespeople: "James" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

v. Replace Martha with Lisa in the salesperson list of sales no. 5.

**Solution:**

```
db.sales.updateOne(
```

```
  { _id: 5, salespeople: "Martha" },
```

```
  { $set: { "salespeople.$": "Lisa" } }
)
```

```
restaurantsdb> db.sales.updateOne(
...   { _id: 5, salespeople: "Martha" },
...   { $set: { "salespeople.$": "Lisa" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

vi. Delete all the sales from California.

**Solution:**

```
db.sales.deleteMany(
```

```
  { state: "CA" }
)
```

```
restaurantsdb> db.sales.deleteMany(
...     { state: "CA" }
... )
{ acknowledged: true, deletedCount: 3 }
```

(c) Print the final sales collection.

**Solution:**

**db.sales.find().pretty()**

```
restaurantsdb> db.sales.find().pretty()
[
  {
    _id: 2, city: 'Bend', state: 'OR', qty: 541 },
  {
    _id: 4, city: 'Eugene', state: 'OR', qty: 892 },
  {
    _id: 5,
    city: 'Reno',
    state: 'NV',
    qty: 655,
    salespeople: [ 'David', 'Lisa', 'James' ]
  },
  {
    _id: 6, city: 'Portland', state: 'OR', qty: 458 }
]
```

3. Load the file books.json into a MongoDB database. Write a Python script that lets the user enter a books category and prints the ISBN and titles of all the books in that category. A sample run:

Enter category : Computer Graphics

ISBN : 1884777902 , title : 3 D User Interfaces with Java 3 D

ISBN : 133034054 , title : Graphics File Formats

ISBN : 1933988398 , title : Gnuplot in Action

ISBN : 1884777473 , title : The Awesome Power of Direct3D / DirectX

ISBN : 138412146 , title : Power -3 D

ISBN : 1930110022 , title : Graphics Programming with Perl

**Solution:**

**from pymongo import MongoClient**

**client = MongoClient("mongodb://localhost:27017/")**

**db = client["booksdb"]**

**books\_collection = db["books"]**

**categories = books\_collection.distinct("categories") # Get all available categories**

**print("Available categories:", categories)**



```
category = input("Enter category: ") # Prompt the user for a category
```

```
# Query the books collection by category
```

```
books = books_collection.find(
    {"categories": {"$in": [category]}},
    {"ISBN": 1, "isbn": 1, "title": 1, "_id": 0}
)
```

```
found = False
```

```
print(f"Books in category '{category}':")
```

```
for book in books:
```

```
    isbn = book.get('ISBN') or book.get('isbn', 'N/A')
```

```
    title = book.get('title', 'N/A')
```

```
    print(f"ISBN: {isbn}, Title: {title}")
```

```
    found = True
```

```
if not found:
```

```
    print("No books found in this category.")
```

```
Available categories: ['', '.NET', 'Algorithmic Art', 'Business', 'Client Server', 'Client-Server', 'Computer Graph', 'C
omputer Graphics', 'In Action', 'Internet', 'Java', 'Microsoft', 'Microsoft .NET', 'Microsoft/.NET', 'Miscella', 'Miscel
laneous', 'Mobile', 'Mobile Technology', 'Networking', 'Next Generation Databases', 'Object-Oriented Programming', 'Obje
ct-Technology Programming', 'Open Source', 'P', 'PHP', 'Perl', 'PowerBuilder', 'Programming', 'Python', 'S', 'SOA', 'Sof
tware Development', 'Software Engineering', 'Theory', 'Web Development', 'XML', 'internet', 'java']
Enter category: Computer Graphics
Books in category 'Computer Graphics':
ISBN: 1884777902, Title: 3D User Interfaces with Java 3D
ISBN: 133034054, Title: Graphics File Formats
ISBN: 1923988398, Title: Gnuplot in Action
ISBN: 1884777473, Title: The Awesome Power of Direct3D/DirectX
ISBN: 138412146, Title: Power-3D
ISBN: 1930110022, Title: Graphics Programming with Perl
Press any key to continue . . . |
```

## 2 MapReduce (20%)

Describe how to implement the following relational operations using MapReduce. Write the map and reduce functions in pseudocode.

1. Projection  $\pi_S(R)$ : From each tuple of relation  $R$  produce only the components for the attributes in  $S$ .

**Solution:**

**# Map function**

**def map(tuple):**

```
projected_tuple = {attr: tuple[attr] for attr in S}    # Extract only the attributes in S
emit(null, projected_tuple)
```

**# Reduce function**

```
def reduce(key, values):
    for value in values:
        emit(value)
```

2. Intersection  $R \cap S$ : Return the tuples that are present in both relations R and S. Assume that relations R and S have the same schema (same attributes and same type).

**Solution:**

**# Map function**

```
def map(tuple, relation_name):
    emit(tuple, relation_name)
```

**# Reduce function**

```
def reduce(tuple, relation_names):
    if "R" in relation_names and "S" in relation_names:
        emit(tuple)
```

3. Grouping  $\gamma_{A, \theta(B)}(R)$ .

Given a relation  $R(A, B, C)$ , with one grouping attribute A, one aggregated attribute B, and another attribute C, which is neither grouped or aggregated:

- (a) Partition the tuples of R according to their values in attribute A.
- (b) For each group, aggregate the values in attribute B and apply function  $\theta$  on the aggregated value ( $\theta$  is an aggregation operation such as SUM, COUNT or MAX).

The result of this operation is one tuple for each group. That tuple has a component for the grouping attribute A, with the value common to tuples of that group. It also has a component for each aggregation  $\theta(B)$ , with the aggregated value for that group

**Solution:**

**# Map function**

**def map(tuple):**

**emit(tuple["A"], tuple["B"])** # Extract only the attributes in S

**# Reduce function**

**def reduce(a\_value, b\_values):**

**aggregated\_value =  $\theta$ (b\_values)** #Apply aggregation function  $\theta$  on the list of B values

**emit({"A": a\_value, " $\theta$ (B)": aggregated\_value})**